



PROGRAMMATION FONCTIONNELLE via le langage Haskell

A. OUALI

L2 au département Mathématique-Informatique
UPR des sciences
Université de Caen Normandie



Les nombres entiers : types `Int` et `Integer`



- opérateurs infixes : `+`, `-`, `*`
- opérateurs préfixes : `div`, `mod`
- précéence et associativité
- comparateurs : `<`, `<=`, `==`, `/=`, `>`, `>=`
- type `Int` (précision limitée) vs type `Integer` (précision infinie)
- le `"-"` peut être utilisé pour représenter la soustraction ou le signe de négativité
 - ➔ Pour éviter la confusion utiliser les parenthèses quand il s'agit d'un nombre négatif

A. Ouali

Programmation fonctionnelle en Haskell

2 / 37

Types élémentaires : entiers `Int`



▸ Nombres Entiers (types `Int` et `Integer`)

```
> 6*7 ==> 42
> 7*4 + 3 ==> 31
> div 13 4 ==> 3
> mod 13 4 ==> 1
```

A. Ouali

Programmation fonctionnelle en Haskell

3 / 37

Les nombres flottants : types `Float` et `Double`



- opérateurs infixes : `+`, `-`, `*`, `/`,
- précéence et associativité
- comparateurs : `<`, `<=`, `==`, `/=`, `>`, `>=`
- type `Float` (simple précision) vs type `Double` (double précision)
- les nombres flottants sont des **approximations** des nombres réels

A. Ouali

Programmation fonctionnelle en Haskell

4 / 37

Types élémentaires : flottants `Float`



▸ Nombres Flottants (types `Float` et `Double`)

```
> sqrt 65 ==> 8.06225774829855
> pi ==> 3.141592653589793
> sin (pi/2) ==> 1.0
> cos (pi/4) ==> 0.7071067811865476
```

A. Ouali

Programmation fonctionnelle en Haskell

5 / 37

Les nombres flottants : types `Float` et `Double`



```
--script
shouldBeTheSame :: Double -> Bool
shouldBeTheSame n = (sqrt n) * (sqrt n) == n

--session
> shouldBeTheSame 1 ==> True
> shouldBeTheSame 10 ==> False
> shouldBeTheSame 100 ==> True
> shouldBeTheSame 100000 ==> False

> shouldBeTheSame 0.1 ==> True
> shouldBeTheSame 0.01 ==> False
> shouldBeTheSame 0.0001 ==> True
> shouldBeTheSame 0.000001 ==> True
> shouldBeTheSame 0.0000001 ==> False
```

En conséquence, **ne jamais tester l'égalité de réels** représentés par des nombres flottants.

A. Ouali

Programmation fonctionnelle en Haskell

6 / 37

Les nombres flottants : types `Float` et `Double`



- ne jamais tester l'égalité de réels représentés par des nombres flottants
- égalité selon une précision voulue

```
estIdentiqueBis :: Double -> Bool
estIdentiqueBis n = abs ((sqrt n) * (sqrt n) - n) < 1/10^10

> estIdentiqueBis 1 ==> True
> estIdentiqueBis 100 ==> True
> estIdentiqueBis 1000 ==> True

> estIdentiqueBis 0.1 ==> True
> estIdentiqueBis 0.01 ==> True
> estIdentiqueBis 0.0001 ==> True
> estIdentiqueBis 0.0000001 ==> True
```

A. Ouali

Programmation fonctionnelle en Haskell

7 / 37

Les booléens : type `Bool`



- seulement 2 valeurs : `True` et `False`
- opérateurs infixes : `&&`, `||`
- opérateurs préfixe : `not`

A. Ouali

Programmation fonctionnelle en Haskell

8 / 37

Types élémentaires : booléen Bool



► Booléens (type Bool)

```
> True && False ==> False
> True || False ==> True
> 2 == 1+1 ==> True
> 5 > 3*4 ==> False
```

Les caractères : type Char



- exemples : 'a', '1', ',', ' ', ''
- caractères spéciaux : tab = '\t', newline = '\n', backslash = '\\'
- relation d'ordre sur les Char
 - fonctions succ et pred de type (Char -> Char)
 - ainsi que les comparateurs : <, <=, ==, /=, >, >=

Types élémentaires : Caractères Char



► Caractères (type Char)

```
> 'a' ==> 'a'
> 'A' < 'a' ==> True
> succ 'b' ==> 'c'
> pred 'z' ==> 'y'
```

Listes



- Liste d'entiers : type [Int]

```
> [1..5] ==> [1,2,3,4,5]
> [1,4..10] ==> [1,4,7,10]
> [2+3, 7-4, div 72 7, 23] ==> [5,3,10,23]
```
- Liste de booléens : type [Bool]

```
> [1 < 2, True, 'a'=='b'] ==> [True,True,False]
```
- ++ : concatène deux listes.
- :: ajoute un élément en tête d'une liste.
- !! : accède à un élément d'une liste en utilisant son index.

Listes



- Liste de caractères : type String

```
> "hello " ++ "world" ==> "hello world"
> ['h', 'e', 'l', 'l', 'o'] ==> "hello"
```
- String et [Char] sont des types synonymes

```
> "hello" == ['h', 'e', 'l', 'l', 'o'] ==> True
> "hello" == ['h', 'e', 'l', 'l', 'o', ' ', ' '] ==> True
```

Tuples



- Tuples

```
> (2^5, True || False, 'a') ==> (32, True, 'a')
> (pi/2, 'A' < 'a') ==> (1.5707963267948966, True)
> ("Lundi", 9, "Janvier", 2018)
==> ("Lundi", 9, "Janvier", 2018)
```

Priorité



- Voici quelques exemples opérateurs courants et leur ordre de priorité en Haskell :
1. Application de fonction : Priorité la plus élevée, associativité à gauche.
 2. Exponentiation (^, ^^, **) : Associativité à droite.
 3. Multiplication et division (*, /, div, mod) : Associativité à gauche.
 4. Addition et soustraction (+, -) : Associativité à gauche.
 5. Opérateur Cons (":") : - Associativité à droite.
 6. Concaténation de listes (++) : - Associativité à droite.
 7. Opérateurs de comparaison (==, /=, <, >, <=, >=) : - Non-associatifs (ne peuvent pas être chaînés).
 8. Opérateurs booléens (&&, ||) : - Associativité à droite.
 9. Composition de fonctions (".") : associativité à droite.
 10. Opérateur d'application (\$) : priorité la plus basse, associativité à droite.

Définir une fonction



- Les noms de fonctions et de paramètres doivent commencer par une lettre minuscule, par exemple myf un1, arg_x, personName, etc.
 - ➔ par convention, un argument de type liste contient 's' un suffixe s dans son nom, p.ex. xs, ns, nss, etc.)
- Une fonction est définie par une égalité :

```
carré x = x * x
plus x y = x + y
```
- Dès que la fonction est définie, on peut l'appliquer par un appel de son nom et les arguments séparés par des espaces :
 - ➔ ce qui est différent dans d'autres langages, p.ex. en langage C : carré(5); et plus(3,2);

```
> carré 5
25
> plus 3 2
5
```
- Les parenthèses sont aussi nécessaires dans Haskell
 - ➔ elles ont l'ordre de priorité le plus élevé, mais servent aussi à délimiter l'appel d'une fonction, p.ex.

```
> plus (carré 5) (carré 3)
34
```

Définition de fonctions



- L'application de fonction a la priorité la plus élevée par rapport à tout
 - ⇒ carré 5 + 3 signifie (carré 5) + 3
- L'appel de fonction s'associe à gauche et se fait par **pattern-matching**
 - ⇒ le premier qui correspond est utilisé
- L'opérateur d'application de fonction \$ a la priorité la plus faible
 - ⇒ utilisé pour supprimer les parenthèses
 - sum ([1..5] ++ [6..10]) sum \$ [1..5] ++ [6..10]

Définir une fonction



- **Factorielle d'un entier**

```
fact :: Int -> Int
fact n = if (n == 0) then 1 else n*(fact (n-1))
```
- **Nombre de chiffres représentant un nombre entier**

```
nbDigits :: Int -> Int
nbDigits n = if (n <= 9) then 1 else 1 + (nbDigits (div n 10))
```

Définir un opérateur



- Des combinaisons de la plupart des symboles sont autorisées comme opérateur
- 4 \$ % ^ & * - + # 23

```
x +/- y = (x+y, x-y)
> 22 +/- 90
(112,-68)
```

Application de fonctions



- En mathématique,
 - l'application de fonction est utilisée avec des parenthèses autour des paramètres en entrée
 - la multiplication est dénoté par juxtaposition ou un espace
- $f(a,b) + c d$
- Dans Haskell,
 - l'application de la fonction est dénotée par un espace
 - la multiplication est dénotée par *

Exemples



Mathématiques	Haskell
f(x)	f x
f(x,y)	f x y
f(g(x))	f (g y)
f(x, g(y))	f x (g y)
f(x) g(y)	(f x) * (g y)

Évaluation d'une fonction



Évaluer par une substitution et une réduction :

```
plus x y = x + y; carré x = x * x
plus (carré 2) (plus 2 3)
-- appliquer carré
plus (2 * 2) (plus 2 3)
-- appliquer *
plus 4 (plus 2 3)
-- appliquer inner plus
plus 4 (2 + 3)
-- appliquer +
plus 4 5
-- appliquer plus
4+5
-- appliquer +
9
```

Évaluation d'une fonction



Plusieurs façon d'appliquer les fonctions :

```
head (1:(reverse [2,3,4,5]))
-- appliquer reverse
-- ... many steps omitted here
head ([1,5,4,3,2])
-- appliquer head
1
```

```
head (1:(reverse [2,3,4,5]))
-- appliquer head
1
```

- Ordre d'évaluation ne modifie pas le résultat
- Affecte le calcul (si calcul termine, avec ou sans échec)
- Évaluation paresseuse

Syntaxe de Base



Pour mieux coder, il faut retenir :

- **Valeur** : donnée concrète p.ex. "toto", 10, ...
- **Type** : ensemble de valeurs ayant des propriétés (opérations) communes : Int, Char, ...
- **Expression** : un morceau de code pouvant être calculé 21 * 2
- **Fonction** : expression avec des arguments en entrée
- Quand on parle d'une variable → **nom associé à une expression**

Syntaxe de Base



► Commentaires :

```
1 -- commentaire de fin de ligne (sur une seule ligne)
2 {- commentaire
3   en multiligne {- et imbricable -} -}
```

► les parenthèses servent uniquement à indiquer des priorités :

```
1 f x = 2 * (x - 3)
```

► et non pour évaluer une fonction :

```
1 Prelude> f 4
2 2
```

Quelques mots réservés



Quelques mots réservés à retenir pour le premier TP :
Liaison d'une expression à un nom ("définir les variables")

► let <expressions groupées> in <expression> :

```
1 f = let y = 1+2
2     z = 4+6
3     in y+z
```

► where :

```
1 f = y+z
2   where y = 1+2
3         z = 4+6
```

► let ... in ... : est une expression contrairement au where qui est réservé pour faire une construction syntaxique

Quelques mots réservés



Quelques mots réservés à retenir pour le premier TP :

► if expression then expression else expression : contrairement au langage impératif le else est obligatoire

```
1 plusPetit x y = if (x < y) then x
2               else y
```

► case of :

```
1 troisPremiers x = case x of
2                   1 -> "A"
3                   2 -> "B"
4                   3 -> "C"
5                   _ -> "*" -- pattern-matching pour le choix par défaut (voir plus tard)
```

Expressions conditionnelles



► if expression then expression else expression : contrairement au langage impératif le else est obligatoire

```
1 dansIntervalle z x y = if (z < x) && (z < y) then -1
2                       else if (x < z) && (z < y) then 0
3                       else if (x < z) && (y < z) then 1
4                       else -2
```

► Utilisation des gardes :

```
1 dansIntervalle z x y
2   | (z < x) && (z < y) = -1
3   | (x < z) && (z < y) = 0
4   | (x < z) && (y < z) = 1
5   | otherwise = -2
```

Syntaxe de Base : indentation



- Le code qui fait partie d'une **expression** doit être indenté plus loin que le début de cette **expression**
 ─ même si l'expression n'est pas l'élément le plus à gauche de la ligne
- La définition doit commencer exactement dans la même colonne (valable aussi pour les expressions groupés)

```
1 a = b + c
2   where
3     b = 1
4     c = 2
5 d = a * 2
```

► Groupement explicite

```
1 a = b + c
2   where
3     {b = 1;
4      c = 2}
5 d = a * 2
```

Expressions conditionnelles (gardes et where)



► Fonction qui retourne une appréciation sur un salaire annuel. La fonction prend en entrée le taux par heure et le nombre d'heures de travail dans une semaine et retourne une chaîne de caractère.

► Définir une fonction annualNetSalaryCalc en utilisant uniquement la syntaxe de gardes et une autre en utilisant des liaisons locales "variables locales".

► Formule : TarifHeure * (HeuresTravailSemaine * 52)
► Intervalles = (20000, 50000, 90000)

Signature (ou type) de fonction



- Un type peut être défini comme une collection de valeurs
- Une signature (ou type d'une fonction) définit les types utilisés pour une fonction
- Généralement écrite dans le format suivant :

```
1 functionName :: arg1Type -> arg2Type -> ... -> resType
```

- Elle n'est pas obligatoire, mais recommandée pour mieux définir la fonction
- On peut utiliser tout type défini dans Haskell
- mais que signifie cette signature type?

```
1 Prelude> :t head
2 head :: [a] -> a
```

Types polymorphes



- Une expression est polymorphe quand elle peut servir, sans modifications, dans des contextes différents.
- Du grec, « poly » = plusieurs et « morphê » = formes.
- **Intérêt** : écrire une seule fonction qui prend en argument des valeurs appartenant de plusieurs types
- Rendre une fonction plus générale pour une application sur différentes données
- Une **variable de type** identifie un type quelconque (ici a)
- Un type est **polymorphe** comporte une variable de type

```
1 f :: a -> a
2 g :: a -> b
```

Types polymorphes



► Définir la fonction retourner

```
1 Prelude>retourner 1
2 1
3 Prelude>retourner "texte"
4 "texte"
```

► Définir la fonction échanger

```
1 Prelude>échanger (1,2)
2 (2, 1)
3 Prelude>échanger (1, "texte")
4 ("texte", 1)
```

► Définir la fonction récupérer

```
1 Prelude>récupérer 2 [1,2,34,8]
2 34
3 Prelude>récupérer 0 [1,2,34,8]
4 1
```

Composition de fonctions



► Définir une fonction qui vérifie si un nombre est impair en utilisant l'opérateur de composition et les fonctions standards (not et even)

```
1 Prelude>impair 3
2 True
```

► Définir une fonction qui trie une liste donnée en entrée avec un ordre décroissant en utilisant l'opérateur de composition

```
1 Prelude>decroissant [1,7,8,5,4,2,4,7]
2 [8,7,7,5,4,4,2,1]
```

Prelude : quelques fonctions utiles



► Sélectionner les n premiers éléments d'une liste :

```
Prelude>take 3 [1,2,3,4,5]
[1,2,3]
```

► Supprimer les n premiers éléments d'une liste :

```
Prelude>drop 3 [1,2,3,4,5]
[4,5]
```

► Inverser une liste :

```
Prelude> reverse [1,2,3,4,5]
[5,4,3,2,1]
```

► Calculer la longueur d'une liste :

```
Prelude> length [1,2,3,4,5]
5
```

► Calculer la somme d'une liste :

```
Prelude> sum [1,2,3,4,5]
15
```

► Calculer le produit d'une liste :

```
Prelude>product [1,2,3,4,5]
120
```

Composition de fonctions



En mathématiques, la composition est le plus souvent représentée par l'opérateur $\circ$, où $f \circ g$ (souvent lu comme f de g) est la composition de f avec g.

Dans haskell, le type de l'opérateur . est :

```
1 (.) :: (b -> c) -> (a -> b) -> a -> c
```

► Composition de fonctions : opérateur infixe (.)

```
1 quadBis :: Int -> Int
2
3 quadBis = carré . carré
4
```

► Carré du carré

```
1 quad :: Int -> Int
2
3 quad x = carré (carré x)
4
```

Prelude : quelques fonctions utiles



- Haskell est diffusé avec un grand nombre de fonctions dans la bibliothèque standard
- Fonctions numériques familières telles que + et *
- De nombreuses fonctions utiles sur les listes

► Sélectionner le premier élément d'une liste :

```
Prelude>head [1,2,3,4,5]
1
```

► Supprimer le premier élément d'une liste :

```
Prelude>tail [1,2,3,4,5]
[2,3,4,5]
```

► Sélectionner le nème élément d'une liste :

```
Prelude>[1,2,3,4,5] !! 2
3
```

► Sélectionner tous les éléments d'une liste sauf le dernier.

```
Prelude>init [1, 2, 3]
[1, 2]
```

► Sélectionner le dernier élément d'une liste.

```
Prelude>last [1, 2, 3]
[3]
```